Course Name:	Web Development Laboratory (316U01L306)	**Semester:**	**III**
Date of Performance:	**__23_ / __9_ / ___26___**	**DIV/ Batch No:**	**B - 2**
Student Name:	**Ishaan Singh Khanka**	**Roll No:**	**16010125107**

Experiment No: 5

Title: JavaScript Programming for Climate Intelligence – Heatwave Monitoring, Prediction, and Early Warning

Aim of the Experiment: To develop JavaScript programs for a Climate Intelligence system that monitors heatwave conditions, analyzes environmental information, determines heatwave risk, and provides early-warning recommendations using JavaScript concepts such as variables, operators, expressions, control statements, objects, functions, arrays, strings, and regular express

Objectives for the Experiment:

- To use JavaScript variables, values, operators, and expressions for processing climate-related information.
- To apply control statements for determining heatwave risk and generating appropriate warnings.
- To create JavaScript objects representing climate monitoring information.
- To use functions for performing climate-related calculations and generating recommendations.
- To use arrays for storing and analyzing temperature and environmental readings.
- To use strings and regular expressions for validating location and user information.
- To integrate JavaScript with HTML forms for a Climate Intelligence early-warning application.

COs to be achieved:

CO1: Developing webpages using HTML, CSS and JavaScript.

Books/ Journals/ Websites references:

1. MDN Web Docs – JavaScript Guide and HTML Forms.
2. W3Schools – JavaScript.

Theory:

Task 1: Heatwave Temperature Analysis

Problem Statement

Develop a JavaScript program to accept the current temperature and threshold temperature of a location and determine the heatwave condition.

Functional Requirements

Declare variables for:

- Location name
- Current temperature
- Heatwave threshold temperature
- Humidity
- Monitoring date

Perform the following operations:

1. Calculate the difference between the current temperature and heatwave threshold.
2. Determine whether the current temperature has crossed the threshold.
3. Display the heatwave monitoring status.
4. Display the location and temperature information.
5. Use JavaScript message-printing methods such as:
    - alert()
    - document.write()
    - console.log()

Expected Output

Location: Mumbai
Current Temperature: 39°C
Heatwave Threshold: 37°C
Temperature Difference: 2°C

Status: Heatwave Condition Detected

Concepts Tested

- Values
- Variables

- Data types
- Operators
- Expressions
- Arithmetic operations
- Message printing
- Basic conditional statement

Deliverable

heatwave_analysis.html

Task 2: Heatwave Risk and Early-Warning Decision

Problem Statement

Develop a JavaScript program that recommends the appropriate heatwave warning level based on temperature, humidity, and forecast conditions.

Input Parameters

- Current temperature
- Humidity
- Forecast temperature
- Heatwave condition
- Vulnerable population alert requirement

Decision Rules

Condition	Recommendation
Temperature ≥ 40°C	**Severe Heatwave Warning**
Temperature ≥ 38°C and Humidity ≥ 60%	**High Heat Risk – Issue Warning**
Temperature between 35°C and 38°C	**Moderate Heat Risk – Monitor Conditions**
Forecast temperature ≥ 40°C	**Early Warning – Prepare for Heatwave**
Temperature < 35°C	**Normal – Continue Monitoring**

Requirements

1. Accept climate values using JavaScript variables.
2. Use if, else-if, and else statements.
3. Use relational operators such as >, <, >=, and <=.
4. Use logical operators such as && and ||.

5. Display the appropriate heatwave risk level.
6. Display an early-warning recommendation.

Example Output

Location: Pune
Temperature: 39°C
Humidity: 65%

Risk Level: HIGH
Recommendation: Issue Heatwave Warning

Concepts Tested

- Variables
- Operators
- Relational operators
- Logical operators
- Expressions
- if-else-if
- Control statements
- Message printing

Deliverable

heatwave_warning.html

Task 3: Create a JavaScript Object for Climate Monitoring

Problem Statement

Create a JavaScript object representing a location monitored by the Climate Intelligence system.

Object Properties

The climate monitoring object should contain:

- Location name
- City
- Temperature
- Humidity
- Wind speed
- Forecast temperature

- Heatwave risk level
- Alert status
- Monitoring date

Required Methods

Create methods to:

1. Display complete climate information.
2. Calculate the temperature difference from the threshold.
3. Determine the heatwave risk level.
4. Display the early-warning status.
5. Update the alert status.

Example Object

```
let climateData = {
   location: "Mumbai",
   temperature: 39,
   humidity: 65,
   windSpeed: 12,
   forecastTemperature: 41,
   riskLevel: "High",
   alertStatus: "Active",

   displayData: function() {
      console.log("Location: " + this.location);
      console.log("Temperature: " + this.temperature + "°C");
      console.log("Humidity: " + this.humidity + "%");
      console.log("Risk Level: " + this.riskLevel);
   }
};
```

Requirements

1. Create the object using object-literal syntax.
2. Access object properties using the dot operator.
3. Create functions/methods inside the object.
4. Use this to access object properties.
5. Display the climate information on the webpage or console.

Concepts Tested

- Object-oriented JavaScript
- Objects

- Properties
- Methods
- Functions
- this keyword
- Conditional statements

Deliverable

climate_object.html

Task 4: Analyze Temperature Readings Using Arrays

Problem Statement

A Climate Intelligence system collects temperature readings at regular intervals. Store the readings in a JavaScript array and analyze the temperature conditions.

Consider the following temperature readings:

let temperatureReadings = [34, 36, 38, 39, 41, 40, 37, 35, 39, 42];

Perform the Following Operations

1. Display all temperature readings.
2. Find the minimum temperature.
3. Find the maximum temperature.
4. Calculate the average temperature.
5. Count the number of readings above the heatwave threshold.
6. Count the number of readings equal to or above 40°C.
7. Display all readings that indicate a high heat condition.

Example Output

Temperature Readings:
34 36 38 39 41 40 37 35 39 42

Minimum Temperature: 34°C
Maximum Temperature: 42°C
Average Temperature: 38.1°C
Readings ≥ 40°C: 3
Heatwave Condition Readings: 41, 40, 42

Concepts Tested

- Arrays
- Array traversal
- Loops
- Array methods
- Functions
- Conditional statements
- Mathematical calculations

Deliverable

temperature_analysis.html

Task 5: Validate Climate Monitoring Registration Details

Problem Statement

Develop a JavaScript program to validate the details entered by a user for registering with the Climate Intelligence Heatwave Early-Warning System.

Fields to Validate

- User name
- Mobile number
- Email address
- Location name
- User ID
- PIN/Postal Code

Validation Rules

Field	Validation Rule
User Name	Only alphabets and spaces
Mobile Number	Exactly 10 digits
Email	Valid email format
Location	Alphabets and spaces
User ID	Format: CLM-1234
PIN Code	Exactly 6 digits

Example

User Name: Rahul Sharma
Mobile Number: 9876543210
Email: rahul@gmail.com
Location: Mumbai
User ID: CLM-1234
PIN Code: 400077

Requirements

1. Accept user details using JavaScript.
2. Validate the user name.
3. Validate the mobile number.
4. Validate the email address.
5. Validate the location.
6. Validate the Climate Intelligence User ID.
7. Display appropriate error messages.
8. Display a success message when all fields are valid.

Regular Expression Examples

User Name:

/^[A-Za-z]+$/

Mobile Number:

/^[6-9][0-9]{9}$/

Climate User ID:

/^CLM-[0-9]{4}$/

PIN Code:

/^[0-9]{6}$/

Concepts Tested

- Strings
- String methods
- Regular expressions
- test()

- Functions
- Boolean expressions
- Conditional statements

Deliverable

climate_validation.html

Task 6: Develop a Climate Intelligence Registration and Early-Warning Form

Problem Statement

Develop an HTML-based registration form for users who want to access the **Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning System**.

The form should collect user, location, and alert-preference information and validate the entered data using JavaScript.

Input Fields

Create a registration form containing:

- User Name
- Mobile Number
- Email
- User ID
- Location/City
- PIN/Postal Code
- Age Group
- User Category
- Preferred Alert Channel
- Registration Date

Suggested User Categories

- General Public
- Senior Citizen
- Student
- Outdoor Worker
- Healthcare Worker
- Government/Disaster Management
- Other

Validation Requirements

1. No mandatory field should remain empty.
2. User name must contain only alphabets and spaces.
3. Mobile number must contain exactly 10 digits.
4. Email address must follow a valid format.
5. User ID must follow the pattern:

CLM-1234

6. PIN/Postal Code must contain exactly six digits.
7. User Category must be selected.
8. Preferred Alert Channel must be selected.
9. Location/City must not be empty.
10. Registration date must be entered.
11. Registration date should not be a future date.
12. Appropriate error messages should be displayed beside invalid fields.
13. Invalid fields should be highlighted.
14. The form should not be submitted until all validation rules are satisfied.
15. After successful validation, display:

Registration Successful!

You are registered for Climate Intelligence
Heatwave Monitoring and Early-Warning Alerts.

HTML Elements to Use

Students should use:

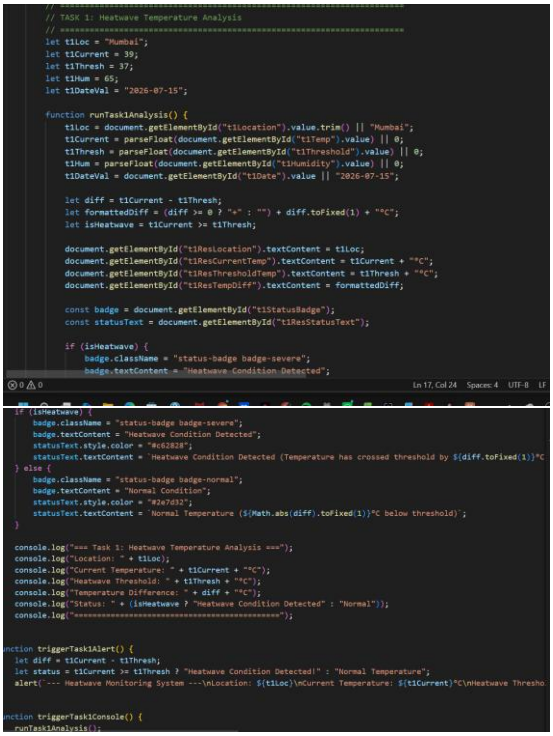
- <form>
- <label>
- <input>
- <select>
- <option>
- <fieldset>
- <legend>
- <button>

JavaScript Requirements

The program should use JavaScript to:

1. Access HTML form elements using the DOM.
2. Read values entered by the user.
3. Validate the input values.
4. Use functions for individual validation operations.
5. Use regular expressions for structured fields.
6. Use conditional statements.
7. Handle the form submit event.
8. Display validation errors.
9. Prevent invalid form submission.
10. Display a success message after valid registration.

Code:



```
// TASK 1: Heatwave Temperature Analysis
// ============================================================
let t1Loc = "Mumbai";
let t1Current = 39;
let t1Thresh = 37;
let t1Hum = 65;
let t1DateVal = "2026-07-15";

function runTask1Analysis() {
    t1Loc = document.getElementById("t1Location").value.trim() || "Mumbai";
    t1Current = parseFloat(document.getElementById("t1Temp").value) || 0;
    t1Thresh = parseFloat(document.getElementById("t1Threshold").value) || 0;
    t1Hum = parseFloat(document.getElementById("t1Humidity").value) || 0;
    t1DateVal = document.getElementById("t1Date").value || "2026-07-15";

    let diff = t1Current - t1Thresh;
    let formattedDiff = (diff >= 0 ? "+" : "") + diff.toFixed(1) + "°C";
    let isHeatwave = t1Current >= t1Thresh;

    document.getElementById("t1ResLocation").textContent = t1Loc;
    document.getElementById("t1ResCurrentTemp").textContent = t1Current + "°C";
    document.getElementById("t1ResThresholdTemp").textContent = t1Thresh + "°C";
    document.getElementById("t1ResTempDiff").textContent = formattedDiff;

    const badge = document.getElementById("t1StatusBadge");
    const statusText = document.getElementById("t1ResStatusText");

    if (isHeatwave) {
        badge.className = "status-badge badge-severe";
        badge.textContent = "Heatwave Condition Detected";
```

```
function triggerTask1Console() {
    runTask1Analysis();
    alert("Parameters logged to Browser Console (Press F12 -> Console to view).");
}

function triggerTask1DocWrite() {
    const container = document.getElementById("t1DocWriteContainer");
    container.style.display = "block";
    const frame = document.getElementById("t1DocWriteFrame");
    const doc = frame.contentWindow.document;
    doc.open();
    let diff = t1Current - t1Thresh;
    let status = t1Current >= t1Thresh ? "Heatwave Condition Detected" : "Normal";
    doc.write("<!DOCTYPE html><html><head><style>body{font-family: Arial; padding: 6px; color:#333;} h4{color:#d84315; m
    doc.write("<h4>document.write() Rendered Output:</h4>");
    doc.write("<p style='margin:2px 0; font-size:12px;'><strong>Location:</strong> " + t1Loc + " | <strong>Current:</str
    doc.write("<p style='margin:2px 0; font-size:12px;'><strong>Status:</strong> <span style='color:red; font-weight:bol
    doc.write("</body></html>");
    doc.close();
}
```

```
<body>
    <script>
        // ==========================================================
        // TASK 2: Heatwave Risk & Early-Warning Decision
        // ==========================================================
        function evaluateTask2Rules() {
            let loc = document.getElementById("t2Location").value.trim() || "Pune";
            let temp = parseFloat(document.getElementById("t2Temp").value) || 0;
            let hum = parseFloat(document.getElementById("t2Humidity").value) || 0;
            let forecast = parseFloat(document.getElementById("t2Forecast").value) || 0;
            let vuln = document.getElementById("t2VulnerableCheck").checked;

            let risk = "";
            let rec = "";
            let recDesc = "";
            let badgeClass = "";
            let borderColor = "";

            if (temp >= 40) {
                risk = "SEVERE";
                rec = "Severe Heatwave Warning";
                recDesc = "Severe heatwave condition detected. Red Alert advisory issued.";
                badgeClass = "status-badge badge-severe";
                borderColor = "#c62828";
            } else if (temp >= 38 && hum >= 60) {
                risk = "HIGH";
                rec = "High Heat Risk ┤ Issue Warning";
                recDesc = "High temperature combined with humidity >= 60%. Issue Heatwave Warning.";
                badgeClass = "status-badge badge-high";
                borderColor = "#e65100";
            } else if (temp >= 35 && temp < 38) {
                risk = "MODERATE";
                rec = "Moderate Heat Risk ┤ Monitor Conditions";
```

```
<html lang="en">
<body>
    <script>
        function evaluateTask2Rules() {
                rec = "Moderate Heat Risk ┤ Monitor Conditions";
                recDesc = "Temperature between 35°C and 38°C. Monitor environment.";
                badgeClass = "status-badge badge-mod";
                borderColor = "#f57f17";
            } else if (forecast >= 40) {
                risk = "EARLY WARNING";
                rec = "Early Warning ┤ Prepare for Heatwave";
                recDesc = "Forecast indicates impending temperature surge above 40°C.";
                badgeClass = "status-badge badge-info";
                borderColor = "#1565c0";
            } else {
                risk = "NORMAL";
                rec = "Normal ┤ Continue Monitoring";
                recDesc = "Temperature < 35°C. Routine climate monitoring.";
                badgeClass = "status-badge badge-normal";
                borderColor = "#2e7d32";
            }

            document.getElementById("t2ResLocation").textContent = loc;
            document.getElementById("t2ResTemp").textContent = temp + "°C";
            document.getElementById("t2ResHumidity").textContent = hum + "%";
            document.getElementById("t2ResForecast").textContent = forecast + "°C";

            const badge = document.getElementById("t2RiskBadge");
            badge.className = badgeClass;
            badge.textContent = risk;

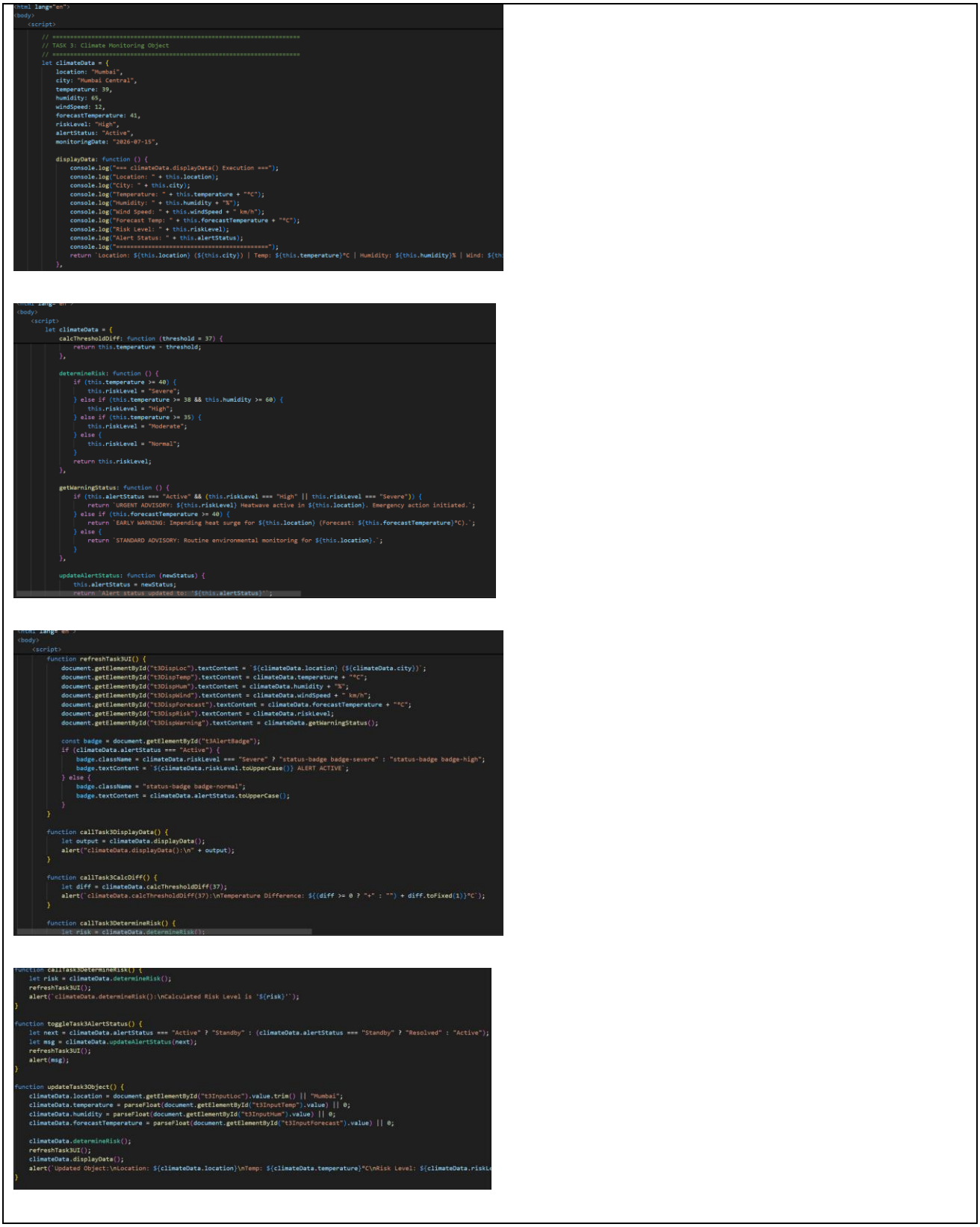
            const recBox = document.getElementById("t2RecBox");
            recBox.style.borderLeftColor = borderColor;
            document.getElementById("t2RecHeading").textContent = "Recommendation: " + rec;
```

```
<html lang="en">
<body>
    <script>
        function evaluateTask2Rules() {
            document.getElementById("t2RecHeading").textContent = "Recommendation: " + rec;
            document.getElementById("t2RecDesc").textContent = recDesc;

            document.getElementById("t2VulnAdvice").style.display = vuln ? "block" : "none";

            console.log("=== Task 2: Heatwave Risk Decision ===");
            console.log(`Location: ${loc} | Temp: ${temp}°C | Humidity: ${hum}% | Risk: ${risk} |
            console.log("=====================================");
        }

        function loadTask2Preset(loc, temp, hum, forecast, vuln) {
            document.getElementById("t2Location").value = loc;
            document.getElementById("t2Temp").value = temp;
            document.getElementById("t2Humidity").value = hum;
            document.getElementById("t2Forecast").value = forecast;
            document.getElementById("t2VulnerableCheck").checked = vuln;
            evaluateTask2Rules();
        }
```



```
<html lang="en">
<body>
    <script>
        // ============================================================
        // TASK 3: Climate Monitoring Object
        // ============================================================
        let climateData = {
            location: "Mumbai",
            city: "Mumbai Central",
            temperature: 39,
            humidity: 65,
            windSpeed: 12,
            forecastTemperature: 41,
            riskLevel: "High",
            alertStatus: "Active",
            monitoringDate: "2026-07-15",

            displayData: function () {
                console.log("=== climateData.displayData() Execution ===");
                console.log("Location: " + this.location);
                console.log("City: " + this.city);
                console.log("Temperature: " + this.temperature + "°C");
                console.log("Humidity: " + this.humidity + "%");
                console.log("Wind Speed: " + this.windSpeed + " km/h");
                console.log("Forecast Temp: " + this.forecastTemperature + "°C");
                console.log("Risk Level: " + this.riskLevel);
                console.log("Alert Status: " + this.alertStatus);
                console.log("========================================");
                return `Location: ${this.location} (${this.city}) | Temp: ${this.temperature}°C | Humidity: ${this.humidity}% | Wind: ${th
            },
```

```
<body>
    <script>
        let climateData = {
            calcThresholdDiff: function (threshold = 37) {
                return this.temperature - threshold;
            },

            determineRisk: function () {
                if (this.temperature >= 40) {
                    this.riskLevel = "Severe";
                } else if (this.temperature >= 38 && this.humidity >= 60) {
                    this.riskLevel = "High";
                } else if (this.temperature >= 35) {
                    this.riskLevel = "Moderate";
                } else {
                    this.riskLevel = "Normal";
                }
                return this.riskLevel;
            },

            getWarningStatus: function () {
                if (this.alertStatus === "Active" && (this.riskLevel === "High" || this.riskLevel === "Severe")) {
                    return `URGENT ADVISORY: ${this.riskLevel} Heatwave active in ${this.location}. Emergency action initiated.`;
                } else if (this.forecastTemperature >= 40) {
                    return `EARLY WARNING: Impending heat surge for ${this.location} (Forecast: ${this.forecastTemperature}°C).`;
                } else {
                    return `STANDARD ADVISORY: Routine environmental monitoring for ${this.location}.`;
                }
            },

            updateAlertStatus: function (newStatus) {
                this.alertStatus = newStatus;
                return `Alert status updated to: '${this.alertStatus}'`;
```

```
<body>
    <script>
        function refreshTask3UI() {
            document.getElementById("t3DispLoc").textContent = `${climateData.location} (${climateData.city})`;
            document.getElementById("t3DispTemp").textContent = climateData.temperature + "°C";
            document.getElementById("t3DispHum").textContent = climateData.humidity + "%";
            document.getElementById("t3DispWind").textContent = climateData.windSpeed + " km/h";
            document.getElementById("t3DispForecast").textContent = climateData.forecastTemperature + "°C";
            document.getElementById("t3DispRisk").textContent = climateData.riskLevel;
            document.getElementById("t3DispWarning").textContent = climateData.getWarningStatus();

            const badge = document.getElementById("t3AlertBadge");
            if (climateData.alertStatus === "Active") {
                badge.className = climateData.riskLevel === "Severe" ? "status-badge badge-severe" : "status-badge badge-high";
                badge.textContent = `${climateData.riskLevel.toUpperCase()} ALERT ACTIVE`;
            } else {
                badge.className = "status-badge badge-normal";
                badge.textContent = climateData.alertStatus.toUpperCase();
            }
        }

        function callTask3DisplayData() {
            let output = climateData.displayData();
            alert("climateData.displayData():\n" + output);
        }

        function callTask3CalcDiff() {
            let diff = climateData.calcThresholdDiff(37);
            alert(`climateData.calcThresholdDiff(37):\nTemperature Difference: ${(diff >= 0 ? "+" : "") + diff.toFixed(1)}°C`);
        }

        function callTask3DetermineRisk() {
            let risk = climateData.determineRisk();
```

```
function callTask3DetermineRisk() {
    let risk = climateData.determineRisk();
    refreshTask3UI();
    alert(`climateData.determineRisk():\nCalculated Risk Level is '${risk}'`);
}

function toggleTask3AlertStatus() {
    let next = climateData.alertStatus === "Active" ? "Standby" : (climateData.alertStatus === "Standby" ? "Resolved" : "Active");
    let msg = climateData.updateAlertStatus(next);
    refreshTask3UI();
    alert(msg);
}

function updateTask3Object() {
    climateData.location = document.getElementById("t3InputLoc").value.trim() || "Mumbai";
    climateData.temperature = parseFloat(document.getElementById("t3InputTemp").value) || 0;
    climateData.humidity = parseFloat(document.getElementById("t3InputHum").value) || 0;
    climateData.forecastTemperature = parseFloat(document.getElementById("t3InputForecast").value) || 0;

    climateData.determineRisk();
    refreshTask3UI();
    climateData.displayData();
    alert(`Updated Object:\nLocation: ${climateData.location}\nTemp: ${climateData.temperature}°C\nRisk Level: ${climateData.riskLe
}
```



```
<body>
    <script>
        // ================================================================
        // TASK 4: Analyze Temperature Readings Using Arrays
        // ================================================================
        const T4_DEFAULT = [34, 36, 38, 39, 41, 40, 37, 35, 39, 42];
        let t4Readings = [...T4_DEFAULT];

        function analyzeTask4Arrays() {
            if (t4Readings.length === 0) return;

            document.getElementById("t4RawArray").textContent = t4Readings.join(", ");

            let min = t4Readings[0];
            let max = t4Readings[0];
            let sum = 0;

            for (let i = 0; i < t4Readings.length; i++) {
                let v = t4Readings[i];
                if (v < min) min = v;
                if (v > max) max = v;
                sum += v;
            }

            let avg = (sum / t4Readings.length).toFixed(1);
            let countGte40 = t4Readings.filter(t => t >= 40).length;
            let heatReadings = t4Readings.filter(t => t >= 40);

            document.getElementById("t4StatCount").textContent = t4Readings.length;
            document.getElementById("t4StatMin").textContent = min + "°C";
            document.getElementById("t4StatMax").textContent = max + "°C";
            document.getElementById("t4StatAvg").textContent = avg + "°C";
```



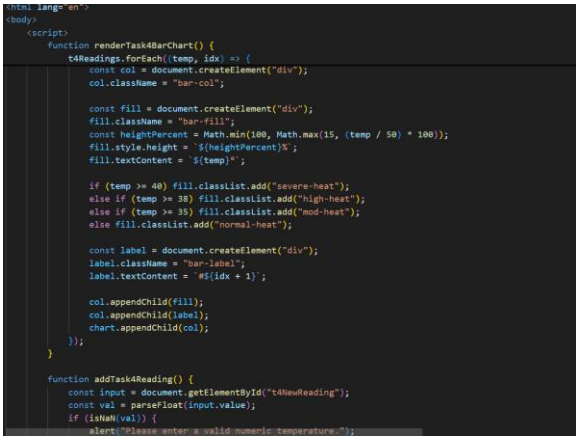
```
<html lang="en">
<body>
    <script>
        function analyzeTask4Arrays() {
            document.getElementById("t4StatMax").textContent = max + "°C";
            document.getElementById("t4StatAvg").textContent = avg + "°C";
            document.getElementById("t4StatGte40").textContent = countGte40;
            document.getElementById("t4HeatWaveList").textContent = heatReadings.length > 0 ? heatReadings.join(", ") : "None";

            const badge = document.getElementById("t4SummaryBadge");
            if (countGte40 >= 3) {
                badge.className = "status-badge badge-severe";
                badge.textContent = `${countGte40} Extreme Readings (≥40°C)`;
            } else if (countGte40 > 0) {
                badge.className = "status-badge badge-high";
                badge.textContent = `${countGte40} High Heat Reading(s)`;
            } else {
                badge.className = "status-badge badge-normal";
                badge.textContent = "Stable Baseline";
            }

            renderTask4BarChart();

            console.log("=== Task 4: Array Processing ===");
            console.log(`Readings: ${t4Readings.join(" ")} | Min: ${min}°C | Max: ${max}°C | Avg: ${avg}°C | >=40°C Count: ${countGte40}`);
            console.log("================================");
        }

        function renderTask4BarChart() {
            const chart = document.getElementById("t4BarChart");
            if (!chart) return;
            chart.innerHTML = "";
```



```
<html lang="en">
<body>
    <script>
        function renderTask4BarChart() {
            t4Readings.forEach((temp, idx) => {
                const col = document.createElement("div");
                col.className = "bar-col";

                const fill = document.createElement("div");
                fill.className = "bar-fill";
                const heightPercent = Math.min(100, Math.max(15, (temp / 50) * 100));
                fill.style.height = `${heightPercent}%`;
                fill.textContent = `${temp}°`;

                if (temp >= 40) fill.classList.add("severe-heat");
                else if (temp >= 38) fill.classList.add("high-heat");
                else if (temp >= 35) fill.classList.add("mod-heat");
                else fill.classList.add("normal-heat");

                const label = document.createElement("div");
                label.className = "bar-label";
                label.textContent = `#${idx + 1}`;

                col.appendChild(fill);
                col.appendChild(label);
                chart.appendChild(col);
            });
        }

        function addTask4Reading() {
            const input = document.getElementById("t4NewReading");
            const val = parseFloat(input.value);
            if (isNaN(val)) {
                alert("Please enter a valid numeric temperature.");
```

```
<html lang="en">
<body>
    <script>
        function addTask4Reading() {
            const input = document.getElementById("t4NewReading");
            const val = parseFloat(input.value);
            if (isNaN(val)) {
                alert("Please enter a valid numeric temperature.");
                return;
            }
            t4Readings.push(val);
            input.value = "";
            analyzeTask4Arrays();
        }

        function popTask4Reading() {
            if (t4Readings.length > 0) {
                let removed = t4Readings.pop();
                analyzeTask4Arrays();
            }
        }

        function resetTask4DefaultArray() {
            t4Readings = [...T4_DEFAULT];
            analyzeTask4Arrays();
        }
```

```
<body>
    <script>
        // ============================================================
        // TASKS 5 & 6: Registration & Regex Validation Form
        // ============================================================
        const t5Form = document.getElementById("t5RegistrationForm");
        const t5Success = document.getElementById("t5SuccessBox");
        const t5Summary = document.getElementById("t5SummaryCard");

        function setTask5Error(fieldId, errorMsg) {
            const input = document.getElementById(fieldId);
            const errorElem = document.getElementById(fieldId + "Error");
            if (!input || !errorElem) return false;

            if (errorMsg) {
                input.classList.remove("form-input-valid");
                input.classList.add("form-input-invalid");
                errorElem.textContent = errorMsg;
                return false;
            } else {
                input.classList.remove("form-input-invalid");
                input.classList.add("form-input-valid");
                errorElem.textContent = "";
                return true;
            }
        }

        function valT5UserName() {
            const val = document.getElementById("t5UserName").value.trim();
            const regex = /^[A-Za-z ]+$/;
            if (val === "") return setTask5Error("t5UserName", "User Name is required.");
            if (val.length < 3) return setTask5Error("t5UserName", "Name must be at least 3 characters.");
            if (!regex.test(val)) return setTask5Error("t5UserName", "Name must contain only alphabets and spaces.");
            return setTask5Error("t5UserName", "");
        }

        function valT5UserId() {
            const val = document.getElementById("t5UserId").value.trim();
            const regex = /^CLM-[0-9]{4}$/;
            if (val === "") return setTask5Error("t5UserId", "User ID is required.");
            if (!regex.test(val)) return setTask5Error("t5UserId", "User ID must follow pattern CLM-1234.");
            return setTask5Error("t5UserId", "");
        }

        function valT5Mobile() {
            const val = document.getElementById("t5Mobile").value.trim();
            const regex = /^[6-9][0-9]{9}$/;
            if (val === "") return setTask5Error("t5Mobile", "Mobile number is required.");
            if (!/^[0-9]{10}$/.test(val)) return setTask5Error("t5Mobile", "Mobile number must be exactly 10 digits.");
```

```
<body>
    <script>
        function valT5Mobile() {
            if (!/^[0-9]{10}$/.test(val)) return setTask5Error("t5Mobile", "Mobile number must be exactly 10 digits.");
            if (!regex.test(val)) return setTask5Error("t5Mobile", "Mobile number must start with 6, 7, 8, or 9.");
            return setTask5Error("t5Mobile", "");
        }

        function valT5Email() {
            const val = document.getElementById("t5Email").value.trim();
            const regex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
            if (val === "") return setTask5Error("t5Email", "Email address is required.");
            if (!regex.test(val)) return setTask5Error("t5Email", "Please enter a valid email format.");
            return setTask5Error("t5Email", "");
        }

        function valT5Location() {
            const val = document.getElementById("t5Location").value.trim();
            const regex = /^[A-Za-z ]+$/;
            if (val === "") return setTask5Error("t5Location", "Location is required.");
            if (!regex.test(val)) return setTask5Error("t5Location", "Location must contain only alphabets and spaces.");
            return setTask5Error("t5Location", "");
        }

        function valT5Pin() {
            const val = document.getElementById("t5Pin").value.trim();
            const regex = /^[0-9]{6}$/;
            if (val === "") return setTask5Error("t5Pin", "PIN code is required.");
            if (!regex.test(val)) return setTask5Error("t5Pin", "PIN code must contain exactly 6 digits.");
            return setTask5Error("t5Pin", "");
        }

        function valT5AgeGroup() {
            const val = document.getElementById("t5AgeGroup").value;
            if (val === "") return setTask5Error("t5AgeGroup", "Please select an age group.");
            return setTask5Error("t5AgeGroup", "");
        }

        function valT5Category() {
            const val = document.getElementById("t5Category").value;
            if (val === "") return setTask5Error("t5Category", "Please select a user category.");
            return setTask5Error("t5Category", "");
        }

        function valT5Channel() {
            const val = document.getElementById("t5Channel").value;
            if (val === "") return setTask5Error("t5Channel", "Please select a preferred alert channel.");
            return setTask5Error("t5Channel", "");
        }
```

```
<html lang="en">
<body>
    <script>
    }

    function valT5RegDate() {
        const val = document.getElementById("t5RegDate").value;
        if (!val) return setTask5Error("t5RegDate", "Registration date is required.");
        const selected = new Date(val);
        const today = new Date();
        today.setHours(23, 59, 59, 999);
        if (selected > today) return setTask5Error("t5RegDate", "Registration date cannot be a future date.");
        return setTask5Error("t5RegDate", "");
    }

    // Attach listeners
    document.getElementById("t5UserName").addEventListener("input", valT5UserName);
    document.getElementById("t5UserId").addEventListener("input", valT5UserId);
    document.getElementById("t5Mobile").addEventListener("input", valT5Mobile);
    document.getElementById("t5Email").addEventListener("input", valT5Email);
    document.getElementById("t5Location").addEventListener("input", valT5Location);
    document.getElementById("t5Pin").addEventListener("input", valT5Pin);
    document.getElementById("t5AgeGroup").addEventListener("change", valT5AgeGroup);
    document.getElementById("t5Category").addEventListener("change", valT5Category);
    document.getElementById("t5Channel").addEventListener("change", valT5Channel);
    document.getElementById("t5RegDate").addEventListener("change", valT5RegDate);

    t5Form.addEventListener("submit", function (e) {
        e.preventDefault();

        const isValid = valT5UserName() && valT5UserId() && valT5Mobile() &&
            valT5Email() && valT5Location() && valT5Pin() &&
            valT5AgeGroup() && valT5Category() && valT5Channel() &&
            valT5RegDate();

        if (isValid) {
            (method) Document.getElementById(elementId: string): HTMLElement | null
            const uName = document.getElementById("t5UserName").value.trim();
            const uId = document.getElementById("t5UserId").value.trim();
            const uMob = document.getElementById("t5Mobile").value.trim();
            const uLoc = document.getElementById("t5Location").value.trim();
            const uPin = document.getElementById("t5Pin").value.trim();
            const uCat = document.getElementById("t5Category").value;
            const uChan = document.getElementById("t5Channel").value;

            t5Success.style.display = "block";
            t5Summary.innerHTML = `<strong>Registered Profile:</strong> ${uName} (${uId}) | Contact: ${uMob} | Location: ${uLoc} (${uPin}) | Category: ${uCat} | Aler

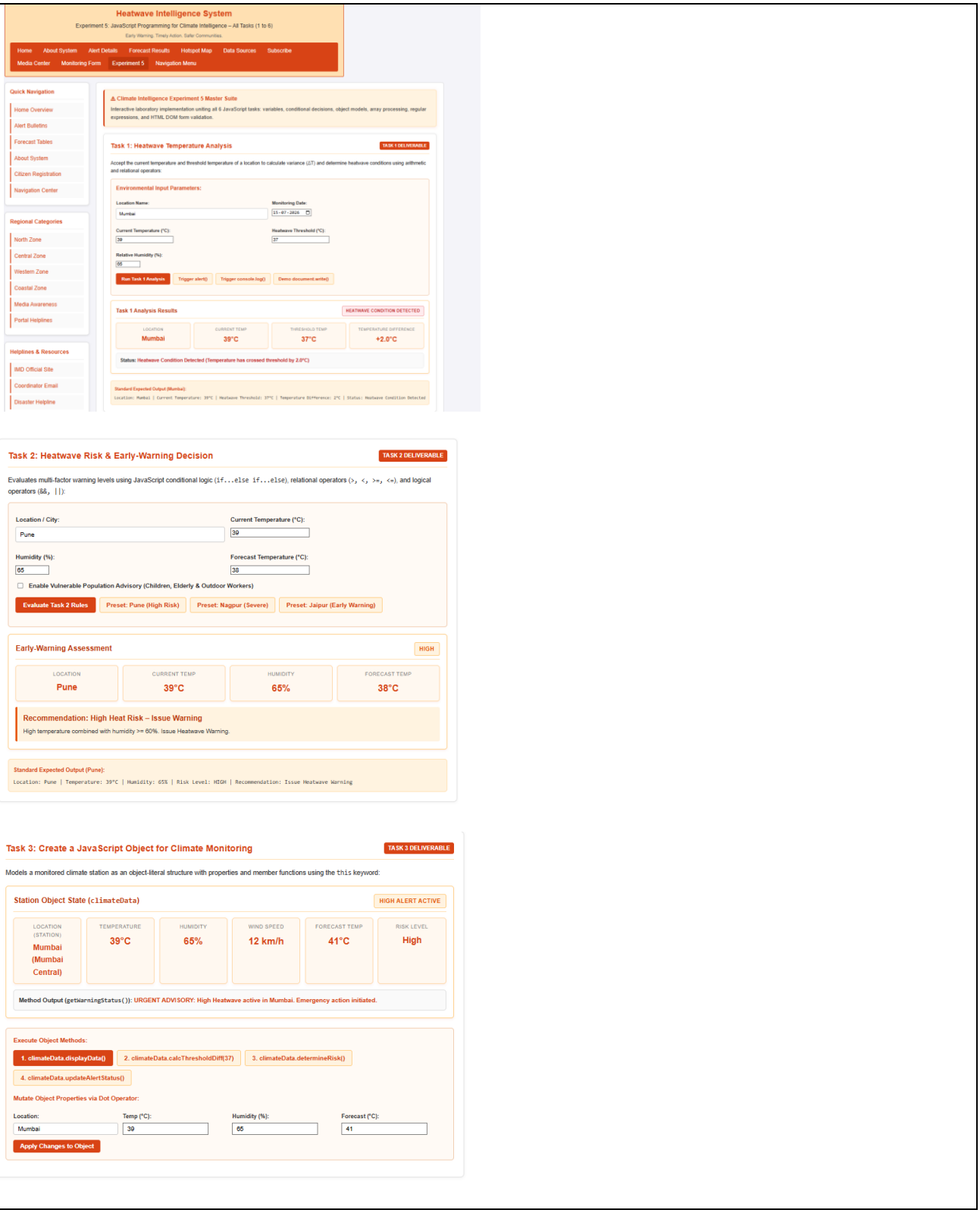
            console.log("=== Tasks 5 & 6: Registration Verified Successfully ===");
            console.log(`User: ${uName} (${uId}) | Mobile: ${uMob} | Location: ${uLoc} - ${uPin}`);
            console.log("=====================================================");

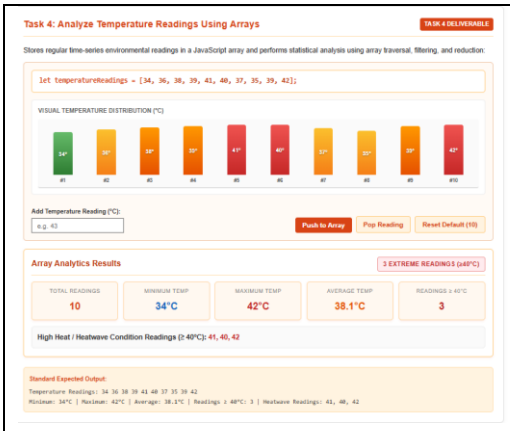
            t5Success.scrollIntoView({ behavior: "smooth" });
        } else {
            t5Success.style.display = "none";
            const firstInvalid = t5Form.querySelector(".form-input-invalid");
            if (firstInvalid) firstInvalid.focus();
        }
    });

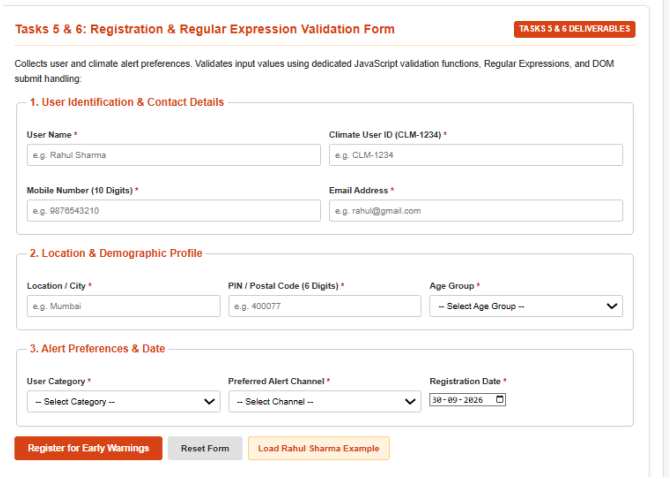
    function fillTask5ValidSample() {
        document.getElementById("t5UserName").value = "Rahul Sharma";
        document.getElementById("t5UserId").value = "CLM-1234";
        document.getElementById("t5Mobile").value = "9876543210";
        document.getElementById("t5Email").value = "rahul@gmail.com";
        document.getElementById("t5Location").value = "Mumbai";
        document.getElementById("t5Pin").value = "400077";
        document.getElementById("t5AgeGroup").value = "Adult (18-59 Years)";
        document.getElementById("t5Category").value = "General Public";
        document.getElementById("t5Channel").value = "SMS Alert";
        document.getElementById("t5RegDate").value = new Date().toISOString().split("T")[0];

        valT5UserName();
        valT5UserId();
        valT5Mobile();
```

Expected Output / Screenshots:(Paste here):


Heatwave Intelligence System
Experiment 5: JavaScript Programming for Climate Intelligence – All Tasks (1 to 6)
Early Warning, Timely Action, Safer Communities.
Home
About System
Alert Details
Forecast Results
Hotspot Map
Data Sources
Subscribe
Media Center
Monitoring Form
Experiment 5
Navigation Menu
Quick Navigation
Home Overview
Alert Bulletins
Forecast Tables
About System
Citizen Registration
Navigation Center
Regional Categories
North Zone
Central Zone
Western Zone
Coastal Zone
Media Awareness
Portal Helplines
Helplines & Resources
IMD Official Site
Coordinator Email
Disaster Helpline
Climate Intelligence Experiment 5 Master Suite
Interactive laboratory implementation uniting all 6 JavaScript tasks: variables, conditional decisions, object models, array processing, regular expressions, and HTML DOM form validation.
Task 1: Heatwave Temperature Analysis
TASK 1 DELIVERABLE
Accept the current temperature and threshold temperature of a location to calculate variance (ΔT) and determine heatwave conditions using arithmetic and relational operators.
Environmental Input Parameters:
Location Name:
Mumbai
Monitoring Date:
15-07-2026
Current Temperature (°C):
39
Heatwave Threshold (°C):
37
Relative Humidity (%):
65
Run Task 1 Analysis
Trigger alert()
Trigger console.log()
Demo document.write()
Task 1 Analysis Results
HEATWAVE CONDITION DETECTED
LOCATION
Mumbai
CURRENT TEMP
39°C
THRESHOLD TEMP
37°C
TEMPERATURE DIFFERENCE
+2.0°C
Status: Heatwave Condition Detected (Temperature has crossed threshold by 2.0°C)
Standard Expected Output (Mumbai):
Location: Mumbai | Current Temperature: 39°C | Heatwave Threshold: 37°C | Temperature Difference: 2°C | Status: Heatwave Condition Detected
Task 2: Heatwave Risk & Early-Warning Decision
TASK 2 DELIVERABLE
Evaluates multi-factor warning levels using JavaScript conditional logic (if...else if...else), relational operators (>, <, >=, <=), and logical operators (&&, ||):
Location / City:
Pune
Current Temperature (°C):
39
Humidity (%):
65
Forecast Temperature (°C):
38
Enable Vulnerable Population Advisory (Children, Elderly & Outdoor Workers)
Evaluate Task 2 Rules
Preset: Pune (High Risk)
Preset: Nagpur (Severe)
Preset: Jaipur (Early Warning)
Early-Warning Assessment
HIGH
LOCATION
Pune
CURRENT TEMP
39°C
HUMIDITY
65%
FORECAST TEMP
38°C
Recommendation: High Heat Risk – Issue Warning
High temperature combined with humidity >= 60%. Issue Heatwave Warning.
Standard Expected Output (Pune):
Location: Pune | Temperature: 39°C | Humidity: 65% | Risk Level: HIGH | Recommendation: Issue Heatwave Warning
Task 3: Create a JavaScript Object for Climate Monitoring
TASK 3 DELIVERABLE
Models a monitored climate station as an object-literal structure with properties and member functions using the this keyword:
Station Object State (climateData)
HIGH ALERT ACTIVE
LOCATION (STATION)
Mumbai (Mumbai Central)
TEMPERATURE
39°C
HUMIDITY
65%
WIND SPEED
12 km/h
FORECAST TEMP
41°C
RISK LEVEL
High
Method Output (getWarningStatus()): URGENT ADVISORY: High Heatwave active in Mumbai. Emergency action initiated.
Execute Object Methods:
1. climateData.displayData()
2. climateData.calcThresholdDiff(37)
3. climateData.determineRisk()
4. climateData.updateAlertStatus()
Mutate Object Properties via Dot Operator:
Location:
Mumbai
Temp (°C):
39
Humidity (%):
65
Forecast (°C):
41
Apply Changes to Object





Conclusion and Discussion:

Students should write the conclusion in their own words based on the work performed.